



COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Unit Convenor)
ptsai@swin.edu.au, EN508d





Week 3



**Writing
MapReduce
Program -I**



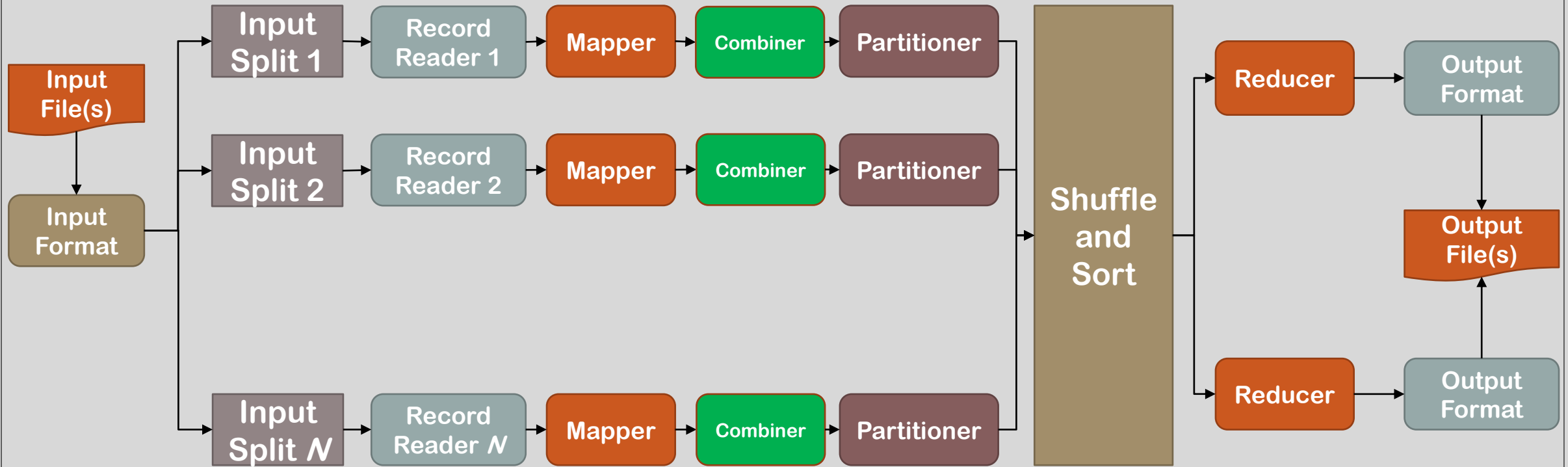
WRITING A MAPREDUCE PROGRAM IN JAVA

Writing a MapReduce Program in Java

Basic MapReduce API concepts

**MapReduce Drivers, Mappers,
and Reducers in Java**

The MapReduce Flow



A Sample MapReduce Program

- You've ran a sample MapReduce program (Word Count) in week 2 lab.
- It counts the number of occurrences of each unique word in a set of files.
- We will examine the code for the Word Count program in this lecture.

The consists of Three Portions

- **Driver code**
 - The code that runs on the client to configure and submit the job.
- **Mapper**
 - The code generates the word and count pairs.
- **Reducer**
 - The code accumulates the counts for every word.

Getting Data to the Mapper

- **The data passed to the Mapper is specified by an InputFormat**
 - Specified in the driver code.
 - Defines the location of the input data. (Typically, a file or a directory.)
 - Determines how to split the input data into input splits. (Each mapper deals with a single input split).
 - Creates a **RecordReader** object for parsing the input data into key-value pairs to pass to the Mapper.

TextInputFormat

- **TextInputFormat** creates the **LineRecordReader** objects. (default)
- It treats each **\n** –terminated line of a file as a value.
- The key is the byte offset of that line within the file.



Tale as old as time, true as it can be.\nBarely even friends then somebody bends,
unexpectedly.\n...

LineRecordReader

Key	Value
0	Tale as old as time, true as it can be.
39	Barely even friends then somebody bends, unexpectedly.
93	...

Other Standard InputFormats

- **FileInputFormat**
 - Abstract base class used for all file-based inputFormats.
- **KeyValueTextInputFormat**
 - Maps \n-terminated lines as “key [separator] value”
 - By default, [separator] is a tab.
- **SequenceFileInputFormat**
 - Binary file of (key,value) pairs with some additional metadata.
- **SequenceFileAsTextInputFormat**
 - Similar, but maps (key.toString(), value.toString())

Keys and Values are Objects

Keys and values in Hadoop are not primitive but Java Objects.

Values are objects which implement *Writable*

Keys are objects which implement *WritableComparable*

Writable

- The *writable* interface makes serialisation quick and easy for Hadoop.
- Any value's type must implement the *Writable* interface.
- Hadoop defines its own “box classes” for strings, integers, and so on.
 - *IntWritable* for integers
 - *LongWritable* for longs
 - *FloatWritable* for floats
 - *DoubleWritable* for doubles
 - *Text* for strings
 - *Etc.*

What is WritableComparable?

- A **WritableComparable** is a **Writable** which is also **Comparable**.
 - Two **WritableComparable**s can be compared against each other to determine their “order.”
 - Keys must be **WritableComparable**s because they are passed to the Reducer in sorted order.
- Despite their names, all Hadoop box classes implement both **Writable** and **WritableComparable**
 - For instance, **IntWritable** is actually a **WritableComparable**.



WRITING A MAPREDUCE PROGRAM IN JAVA



WHAT IS A VIRTUAL MACHINE?

Starting with the Driver Code

The driver code runs on the client machine.

It configures the job, and then submits it to the cluster.

```
package stubs;
```

```
import org.apache.hadoop.fs.Path;  
import org.apache.hadoop.io.IntWritable;  
import org.apache.hadoop.io.Text;  
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;  
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;  
import org.apache.hadoop.mapreduce.Job;
```

DRIVER CODE EXAMPLE


```
package stubs;
```

```
import org.apache.hadoop.fs.Path;  
import org.apache.hadoop.io.IntWritable;  
import org.apache.hadoop.io.Text;  
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;  
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;  
import org.apache.hadoop.mapreduce.Job;
```

DRIVER CODE EXAMPLE

```
public class WordCount {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.printf(
                "Usage: WordCount <input dir> <output dir>\n");
            System.exit(-1);
        }

        Job job = new Job();
        job.setJarByClass(WordCount.class);
        job.setJobName("Word Count");

        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.setMapperClass(WordMapper.class);
        job.setReducerClass(SumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        boolean success = job.waitForCompletion(true);
        System.exit(success ? 0 : 1);
    }
}
```

DRIVER CODE EXAMPLE

```
public class WordCount {  
    public static void main(String[] args) throws Exception {  
        if (args.length != 2) {  
            System.out.printf(  
                "Usage: WordCount <input dir> <output dir>\n");  
            System.exit(-1);  
        }  
  
        Job job = new Job();  
        job.setJarByClass(WordCount.class);  
        job.setJobName("Word Count");  
  
        FileInputFormat.setInputPaths(job, new Path(args[0]));  
        FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
        job.setMapperClass(WordMapper.class);  
        job.setReducerClass(SumReducer.class);  
        job.setOutputKeyClass(Text.class);  
        job.setOutputValueClass(IntWritable.class);  
  
        boolean success = job.waitForCompletion(true);  
        System.exit(success ? 0 : 1);  
    }  
}
```

DRIVER CODE EXAMPLE

```
public class WordCount {  
    public static void main(String[] args) throws Exception {  
        if (args.length != 2) {  
            System.out.printf(  
                "Usage: WordCount <input dir> <output dir>\n");  
            System.exit(-1);  
        }  
    }  
}
```

```
Job job = new Job();  
job.setJarByClass(WordCount.class);  
job.setJobName("Word Count");
```

```
FileInputFormat.setInputPaths(job, new Path(args[0]));  
FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

```
job.setMapperClass(WordMapper.class);  
job.setReducerClass(SumReducer.class);  
job.setOutputKeyClass(Text.class);  
job.setOutputValueClass(IntWritable.class);
```

```
boolean success = job.waitForCompletion(true);  
System.exit(success ? 0 : 1);
```

```
    }  
}
```

DRIVER CODE EXAMPLE

The Job Object

- The job class allows you to set configuration options for your MapReduce job.
- Any options not explicitly set in your driver code or specified in your configuration will be read from your Hadoop configuration files.
 - Usually located in `/etc/hadoop/conf`
- You can also use the Job object to submit the job, control its execution, and query its state.


```
public class WordCount {  
    public static void main(String[] args) throws Exception {  
        if (args.length != 2) {  
            System.out.printf(  
                "Usage: WordCount <input dir> <output dir>\n");  
            System.exit(-1);  
        }  
  
        Job job = new Job();  
        job.setJarByClass(WordCount.class);  
        job.setJobName("Word Count");  
  
        FileInputFormat.setInputPaths(job, new Path(args[0]));  
        FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
        job.setMapperClass(WordMapper.class);  
        job.setReducerClass(SumReducer.class);  
        job.setOutputKeyClass(Text.class);  
        job.setOutputValueClass(IntWritable.class);  
  
        boolean success = job.waitForCompletion(true);  
        System.exit(success ? 0 : 1);  
    }  
}
```

DRIVER CODE EXAMPLE

Specifying the InputFormat

- The default InputFormat (TextInputFormat) will be used unless you specify otherwise.
- To use an InputFormat other than the default, use the following command:
 - `job.setInputFormatClass(KeyValueTextInputFormat.class)`

InputFormat

- By default, `FileInputFormat.setInputPaths()` will read **all files** from a specified directory and send them to Mappers.
 - Exceptions: items with the name beginning with a period (.) or underscore (_).
- Alternatively, `FileInputFormat.addInputPath()` can be used multiple times, specifying a single file/directory each time.

OutputFormat

- **FileOutputFormat.setOutputPath()** specifies the directory to which the Reducers will write their final output.
- **The driver can also specify the format of the output data.**
 - **Default is a plain text file.**
 - **Could be explicitly written as:**
job.setOutputFormatClass(TextOutputFormat.class)

```
public class WordCount {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.printf(
                "Usage: WordCount <input dir> <output dir>\n");
            System.exit(-1);
        }

        Job job = new Job();
        job.setJarByClass(WordCount.class);
        job.setJobName("Word Count");

        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.setMapperClass(WordMapper.class);
        job.setReducerClass(SumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        boolean success = job.waitForCompletion(true);
        System.exit(success ? 0 : 1);
    }
}
```

DRIVER CODE EXAMPLE

```
public class WordCount {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.printf(
                "Usage: WordCount <input dir> <output dir>\n");
            System.exit(-1);
        }

        Job job = new Job();
        job.setJarByClass(WordCount.class);
        job.setJobName("Word Count");

        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.setMapperClass(WordMapper.class);
        job.setReducerClass(SumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        boolean success = job.waitForCompletion(true);
        System.exit(success ? 0 : 1);
    }
}
```

DRIVER CODE EXAMPLE


```
public class WordCount {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.printf(
                "Usage: WordCount <input dir> <output dir>\n");
            System.exit(-1);
        }

        Job job = new Job();
        job.setJarByClass(WordCount.class);
        job.setJobName("Word Count");

        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.setMapperClass(WordMapper.class);
        job.setReducerClass(SumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        boolean success = job.waitForCompletion(true);
        System.exit(success ? 0 : 1);
    }
}
```

DRIVER CODE EXAMPLE

waitForCompletion() vs Submit()

- There are two ways to run your MapReduce job:
 - `job.waitForCompletion()`
 - `job.submit()`
- The client determines the proper division of input data into `InputSplits`, and then sends the job information to the `JobTracker` daemon on the cluster.



```
package stubs;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
```

MAPPER CODE EXAMPLE

```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    @Override  
    public void map(LongWritable key, Text value, Context context)  
        throws IOException, InterruptedException {  
  
        String line = value.toString();  
  
        for (String word : line.split("\\W+")) {  
            if (word.length() > 0) {  
  
                context.write(new Text(word), new IntWritable(1));  
            }  
        }  
    }  
}
```

MAPPER CODE EXAMPLE

```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    @Override  
    public void map(LongWritable key, Text value, Context context)  
        throws IOException, InterruptedException {  
  
        String line = value.toString();  
  
        for (String word : line.split("\\W+")) {  
            if (word.length() > 0) {  
  
                context.write(new Text(word), new IntWritable(1));  
            }  
        }  
    }  
}
```

MAPPER CODE EXAMPLE

```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    @Override  
    public void map(LongWritable key, Text value, Context context)  
        throws IOException, InterruptedException {  
  
        String line = value.toString();  
  
        for (String word : line.split("\\W+")) {  
            if (word.length() > 0) {  
  
                context.write(new Text(word), new IntWritable(1));  
            }  
        }  
    }  
}
```

MAPPER CODE EXAMPLE

```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    @Override  
    public void map(LongWritable key, Text value, Context context)  
        throws IOException, InterruptedException {  
  
        String line = value.toString();  
  
        for (String word : line.split("\\W+")) {  
            if (word.length() > 0) {  
                context.write(new Text(word), new IntWritable(1));  
            }  
        }  
    }  
}
```

MAPPER CODE EXAMPLE


```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    @Override  
    public void map(LongWritable key, Text value, Context context)  
        throws IOException, InterruptedException {  
        String line = value.toString();  
  
        for (String word : line.split("\\W+")) {  
            if (word.length() > 0) {  
                context.write(new Text(word), new IntWritable(1));  
            }  
        }  
    }  
}
```

MAPPER CODE EXAMPLE

```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    @Override  
    public void map(LongWritable key, Text value, Context context)  
        throws IOException, InterruptedException {  
  
        String line = value.toString();  
  
        for (String word : line.split("\\W+")) {  
            if (word.length() > 0) {  
  
                context.write(new Text(word), new IntWritable(1));  
            }  
        }  
    }  
}
```

MAPPER CODE EXAMPLE

```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    @Override  
    public void map(LongWritable key, Text value, Context context)  
        throws IOException, InterruptedException {  
  
        String line = value.toString();  
  
        for (String word : line.split("\\W+")) {  
            if (word.length() > 0) {  
                context.write(new Text(word), new IntWritable(1));  
            }  
        }  
    }  
}
```

MAPPER CODE EXAMPLE



```
package stubs;
```

```
import java.io.IOException;
```

```
import org.apache.hadoop.io.IntWritable;
```

```
import org.apache.hadoop.io.Text;
```

```
import org.apache.hadoop.mapreduce.Reducer;
```

REDUCER CODE EXAMPLE

```
public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {  
    @Override  
    public void reduce(Text key, Iterable<IntWritable> values, Context context)  
        throws IOException, InterruptedException {  
  
        int wordCount = 0;  
  
        for (IntWritable value : values) {  
            wordCount += value.get();  
        }  
  
        context.write(key, new IntWritable(wordCount));  
    }  
}
```

REDUCER CODE EXAMPLE

```
public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {  
    @Override  
    public void reduce(Text key, Iterable<IntWritable> values, Context context)  
        throws IOException, InterruptedException {  
  
        int wordCount = 0;  
  
        for (IntWritable value : values) {  
            wordCount += value.get();  
        }  
  
        context.write(key, new IntWritable(wordCount));  
    }  
}
```

REDUCER CODE EXAMPLE

```
public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {  
    @Override  
    public void reduce(Text key, Iterable<IntWritable> values, Context context)  
        throws IOException, InterruptedException {  
  
        int wordCount = 0;  
  
        for (IntWritable value : values) {  
            wordCount += value.get();  
        }  
  
        context.write(key, new IntWritable(wordCount));  
    }  
}
```

REDUCER CODE EXAMPLE

```
public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {  
    @Override  
    public void reduce(Text key, Iterable<IntWritable> values, Context context)  
        throws IOException, InterruptedException {  
  
        int wordCount = 0;  
  
        for (IntWritable value : values) {  
            wordCount += value.get();  
        }  
  
        context.write(key, new IntWritable(wordCount));  
    }  
}
```

REDUCER CODE EXAMPLE


```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    @Override  
    public void map(LongWritable key, Text value, Context context)  
        throws IOException, InterruptedException {  
  
        String line = value.toString();  
  
        for (String word : line.split("\\W+")) {  
            if (word.length() > 0) {  
                context.write(new Text(word), new IntWritable(1));  
            }  
        }  
    }  
}
```

ENSURE TYPES MATCH

Key Points

InputFormat

- Parses input files into key/value pairs.

WritableComparable/Writable classes

- “Box” or “Wrapper” classes to pass keys and values.

Driver

- Sets InputFormat and input and output types.
- Specifies classes for the Mapper and Reducer.

Mapper

- map() method takes a key/value pair.
- Call Context.write() to output intermediate key/value pairs.

Reducer

- reduce() method takes a key and iterable list of values.
- Call Context.write() to output final key/value pairs.



INTEGRATED DEVELOPMENT ENVIRONMENTS



Integrated Development Environments

- There are many integrated Development Environments (IDEs) available.
- Eclipse is one of the IDEs.
 - Open source.
 - Very popular among Java developers.
 - Has plug-ins to speed up the development in several different languages.



STREAMING

The Streaming API - Motivation

- Many organizations have developers skilled in languages other than Java. (ex: C++, C#, Ruby, Python, Perl, etc.)
- The streaming API allows developers to use any language they wish to write Mappers and Reducers.
 - As long as the language can read from standard input and write to standard output.

Streaming API: Advantages and Disadvantages

Advantages

- No need for non-Java coders to learn Java.
- Fast development time.
- Ability to use existing code libraries.

Disadvantages

- Performance.
- Primarily suited for handling data that can be represented as text.
- Streaming jobs can use excessive amounts of RAM or fork excessive numbers of processes.
- Although Mappers and Reducers can be written using the Streaming API, Partitioners, InputFormats etc. **must** still be written in Java.

How Streaming Works?

To implement streaming, write separate Mapper and Reducer programs in the language(s) of your choice.

- They will receive input via *stdin*.
- They should write their output to *stdout*.

If *TextInputFormat* (the default) is used, the streaming Mapper just receives each line from the file on *stdin*.

- No key is passed.

Mapper and Reducer output should be sent to *stdout* as:

- *Key [tab] value [newline]*

Separators other than tab can be specified.


```
#!/usr/bin/env python
"""mapper.py"""

import sys

# input comes from STDIN (standard input)
for line in sys.stdin:
    # remove leading and trailing whitespace
    line = line.strip()
    # split the line into words
    words = line.split()
    # increase counters
    for word in words:
        # write the results to STDOUT (standard output);
        # what we output here will be the input for the
        # Reduce step, i.e. the input for reducer.py
        #
        # tab-delimited; the trivial word count is 1
        print '%s\t%s' % (word, 1)
```

PYTHON MAPREDUCE EXAMPLE

Mapper code

```
#!/usr/bin/env python
"""reducer.py"""

from operator import itemgetter
import sys

current_word = None
current_count = 0
word = None

# input comes from STDIN
for line in sys.stdin:
    # remove leading and trailing whitespace
    line = line.strip()

    # parse the input we got from mapper.py
    word, count = line.split('\t', 1)

    # convert count (currently a string) to int
    try:
        count = int(count)
    except ValueError:
        # count was not a number, so silently
        # ignore/discard this line
        continue

    # this IF-switch only works because Hadoop sorts map output
    # by key (here: word) before it is passed to the reducer
    if current_word == word:
        current_count += count
    else:
        if current_word:
            # write result to STDOUT
            print '%s\t%s' % (current_word, current_count)
            current_count = count
            current_word = word

# do not forget to output the last word if needed!
if current_word == word:
    print '%s\t%s' % (current_word, current_count)
```

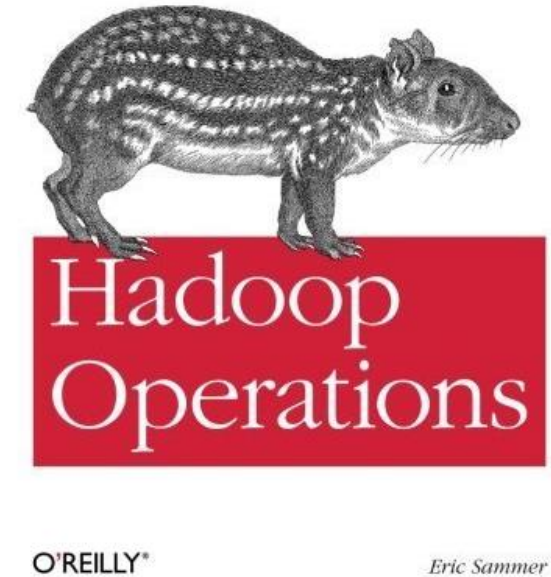
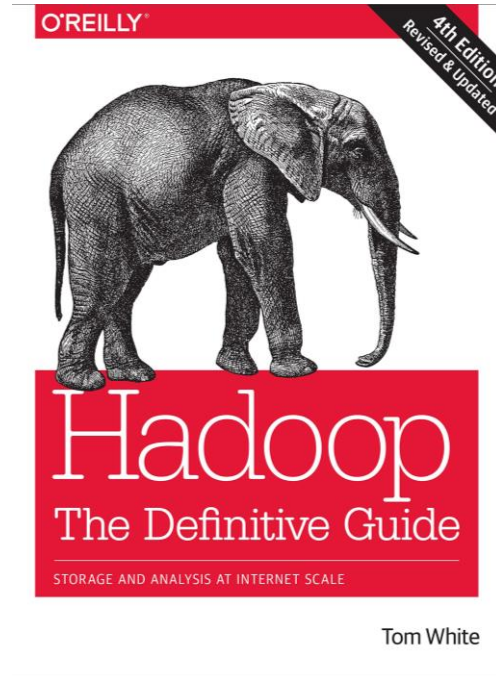
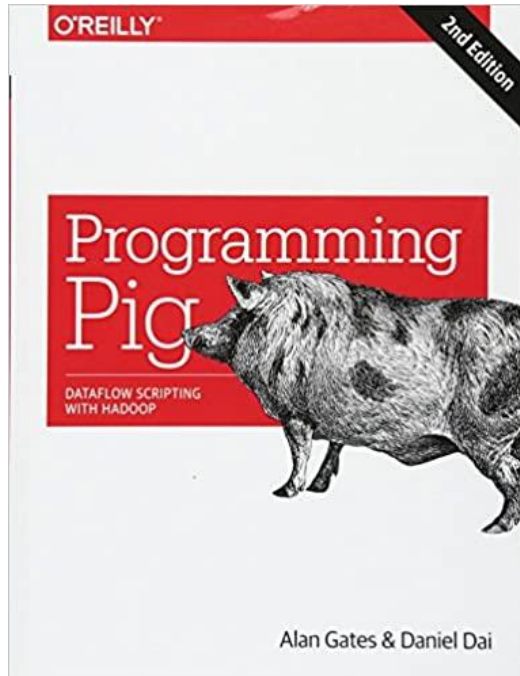
PYTHON MAPREDUCE EXAMPLE

Reducer code

Python MapReduce Example

- **Running the Python Code on Hadoop**

```
hduser@ubuntu:/usr/local/hadoop$ bin/hadoop jar contrib/streaming/hadoop-*streaming*.jar \  
-file /home/hduser/mapper.py -mapper /home/hduser/mapper.py \  
-file /home/hduser/reducer.py -reducer /home/hduser/reducer.py \  
-input /user/hduser/gutenberg/* -output /user/hduser/gutenberg-output
```



Texts and Resources

- Unless stated otherwise, the materials presented in this lecture are taken from:
 - Hadoop: the definitive guide, White, Tom (Tom E.) author., 4th ed., Sebastopol, California: O'Reilly, 2015.
 - Hadoop operations, Sammer, Eric., Loukides, Michael Kosta.; Nash, Courtney.; Romano, Robert (Illustrator), illustrator., First edition., Sebastopol, CA: O'Reilly, 2012.
 - Programming pig: dataflow scripting with hadoop, Gates, Alan, author.; Dai, Daniel, 2nd edition., Beijing, China: O'Reilly, 2017



Unit Summary

- MapReduce Code Explanation.

Summary

